CMPSC 448 Midterm Project Report

Nicholas de Rezende – Group Leader

Mahdi Shah

Problem definition/implementation: Implementation

In this project, we trained a convolutional neural network (CNN) and a recurrent neural network (RNN) to predict which large language model generated a response. We created 50 unique prompts and passed each through ChatGPT, Claude, and Gemini, giving us 150 labeled responses stored in a CSV file.

Before training, we cleaned the dataset using Python and assigned numerical labels to each model: ChatGPT (0), Claude (1), and Gemini (2). We tokenized the text into words and punctuation, then created a vocabulary using the training data. Each token was assigned a numerical ID, and sequences were padded or truncated to a maximum length of 200 tokens for the reported experiments so they could be processed in batches.

For the reported experiments, we split the dataset into 96 training, 24 validation, and 30 testing responses using a fixed random seed. The training data was used to update the models, the validation data was used to monitor performance and guide model selection, and the testing data was used to evaluate the final results.

For our CNN, we used an embedding layer with 128 dimensions, followed by a one-dimensional convolutional layer containing 64 filters with a kernel size of 3. We applied a ReLU activation function and padding-aware max pooling to extract local patterns from the text. The resulting features were passed through a dropout layer with a rate of 0.5 and a fully connected layer to classify responses into three LLM categories. The CNN was trained for 10 epochs using the Adam optimizer, a learning rate of 0.001, cross-entropy loss, and a batch size of 8.

For our RNN, we used a Long Short-Term Memory (LSTM) architecture with an embedding size of 128, a hidden size of 64, and a dropout rate of 0.5 to help prevent overfitting. The LSTM processed the text sequences and passed its final hidden state through a fully connected layer to predict one of the three LLMs. We trained the model for 10 epochs using the Adam optimizer, a learning rate of 0.001, cross-entropy loss, and a batch size of 8.

After training, we evaluated the models using test accuracy, precision, recall, F1-score, and confusion matrices to compare how well they identified responses from each LLM.

RQ1: CNN and RNN Results

```
=== CNN Test Results ===
Test Accuracy: 96.67%
Correct predictions: 29
Total test responses: 30

=== CNN Classification Report ===
              precision    recall  f1-score   support

         GPT       0.91      1.00      0.95        10
      Claude       1.00      1.00      1.00        10
      Gemini       1.00      0.90      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

```
=== LSTM Test Results ===
Test Accuracy: 46.67%
Correct predictions: 14
Total test responses: 30

=== LSTM Classification Report ===
              precision    recall  f1-score   support

         GPT       0.45      0.50      0.48        10
      Claude       0.50      0.60      0.55        10
      Gemini       0.43      0.30      0.35        10

    accuracy                           0.47        30
   macro avg       0.46      0.47      0.46        30
weighted avg       0.46      0.47      0.46        30
```

Data collection and preprocessing

After collecting and preprocessing our dataset, we trained both a convolutional neural network (CNN) and a recurrent neural network (RNN) using an LSTM architecture. We split our 150 responses into 96 training samples, 24 validation samples, and 30 testing samples. For this first experiment, we only used the output responses generated by each LLM to predict which model they came from.

The CNN performed significantly better than the RNN. The CNN achieved a testing accuracy of 96.67%, correctly predicting 29 out of 30 responses. The RNN achieved a testing accuracy of 46.67%, correctly predicting 14 out of 30 responses. Since we were classifying between three

different LLMs, random guessing would give an expected accuracy of approximately 33.33%. Both models performed above this baseline, but the CNN achieved substantially higher accuracy.

Looking at the confusion matrices, the CNN correctly identified all 10 ChatGPT responses and all 10 Claude responses, but incorrectly classified one Gemini response as ChatGPT. The RNN had more difficulty distinguishing between the models, correctly identifying 5 ChatGPT responses, 6 Claude responses, and 3 Gemini responses out of 10 responses per model. The RNN performed best at identifying Claude responses and struggled most with Gemini, suggesting that the CNN was more effective at recognizing differences in the generated text.

```
=== CNN Confusion Matrix ===
[[10  0  0]
 [ 0 10  0]
 [ 1  0  9]]
```

```
=== LSTM Confusion Matrix ===
[[5 2 3]
 [3 6 1]
 [3 4 3]]
```

One interesting finding was that the RNN showed signs of overfitting. By the final epoch, it reached a training accuracy of 96.88%, but its validation accuracy was only 41.67%. This substantial difference suggests that the RNN learned patterns specific to the training responses rather than patterns that generalized effectively to unseen data. The CNN performed considerably better on the test set, suggesting that it was more effective at identifying differences in the writing styles of the three LLMs under our experimental conditions. Possible contributing factors to the RNN's lower performance include the relatively small dataset and the model's training configuration. However, these results do not establish that CNNs are universally better than RNNs.

Our results showed that the CNN was much more effective for this task than the RNN. However, since our dataset only contained 150 responses, we cannot necessarily assume that the CNN would perform this well on a much larger dataset. It would be interesting to see whether these results stay consistent if we trained both models on more examples.

RQ2: Comparing different input configurations

Input only:

```
=== CNN Test Results ===
Test Accuracy: 33.33%
Correct predictions: 10
Total test responses: 30

=== CNN Classification Report ===
              precision    recall  f1-score   support

         GPT       0.33      0.10      0.15        10
      Claude       0.33      0.20      0.25        10
      Gemini       0.33      0.70      0.45        10

    accuracy                           0.33        30
   macro avg       0.33      0.33      0.29        30
weighted avg       0.33      0.33      0.29        30
```

```
=== LSTM Test Results ===
Test Accuracy: 33.33%
Correct predictions: 10
Total test responses: 30

=== LSTM Classification Report ===
              precision    recall  f1-score   support

         GPT       0.33      0.40      0.36        10
      Claude       0.33      0.20      0.25        10
      Gemini       0.33      0.40      0.36        10

    accuracy                           0.33        30
   macro avg       0.33      0.33      0.33        30
weighted avg       0.33      0.33      0.33        30
```

For the second research question, we wanted to see how the models would perform when trained using different combinations of the original prompts and generated responses. We started by training both models using only the original prompts.

Both the CNN and RNN achieved a testing accuracy of 33.33%, correctly predicting 10 out of 30 responses. This was expected because we gave ChatGPT, Claude, and Gemini the exact same 50 prompts. Since every prompt appeared with all three model labels, the input alone did not provide enough information to distinguish between them.

Neither model showed much improvement throughout training, with validation accuracy remaining at 33.33% for all 10 epochs. This suggests that the original prompts were not useful on their own for identifying which LLM generated a response.

Combined Input and Output:

For our final experiment in RQ2, we trained both the CNN and RNN using the original prompts combined with the responses generated by each LLM. We wanted to see whether including the original prompt alongside the generated response would improve the models' ability to predict which LLM produced it.

The CNN achieved a testing accuracy of 76.67%, correctly identifying 23 out of 30 responses. The RNN achieved a testing accuracy of 40.00%, correctly identifying 12 out of 30 responses. While both models performed above the 33.33% accuracy expected from random guessing, neither performed as well as when we used only the generated responses. The CNN's accuracy decreased from 96.67% to 76.67%, while the RNN's accuracy decreased from 46.67% to 40.00%.

One possible explanation for the decrease in accuracy is sequence truncation. Since we limited each sequence to 200 tokens, combining the original prompt with the generated response may have caused some useful response information to be excluded. Additionally, all three LLMs received identical prompts, so the prompts themselves provided no direct information about which model generated a response. However, further experiments would be needed to determine whether truncation was the primary cause of the decreased performance.

Overall, our results showed that combining the prompts and responses did not improve the performance of either model. The CNN still performed significantly better than the RNN, but using only the generated responses gave us the highest accuracy for both models.

Lessons/ Experiences:

Throughout this project, we gained experience with the full machine learning process, from collecting and preprocessing data to training neural networks and evaluating their performance. We also learned how important data preparation is when working with machine learning models.

One challenge we encountered was collecting our dataset. We manually submitted 50 prompts to ChatGPT, Claude, and Gemini, then used a Python script to organize the responses into a CSV file. While this worked, we learned that using APIs could have automated the process and saved time. Our preprocessing script was also important because it converted the raw text into numerical data that could be used by both neural networks.

We also gained a better understanding of the differences between CNNs and RNNs. CNNs identify local patterns, such as common phrases and word combinations, while RNNs process text sequentially to learn relationships between words. In our experiments, the CNN performed much better, achieving 96.67% testing accuracy compared to the RNN's 46.67% when using only generated responses. The RNN also showed signs of overfitting, as its training accuracy increased significantly without similar improvements in validation accuracy. This suggests that the CNN was more effective at identifying patterns in our dataset.

Another interesting finding was that combining the original prompts with the generated responses did not improve classification accuracy. The CNN's accuracy decreased to 76.67%, while the RNN's decreased to 40.00%. One possible reason is that all three LLMs received identical prompts, so the additional text did not provide useful information for distinguishing between them. Additionally, the 200-token sequence limit may have caused some generated-response information to be truncated when the original prompts were included. This demonstrated how preprocessing decisions can influence model performance and the importance of evaluating different input configurations.

Overall, this project helped us understand what it takes to build a machine learning system from start to finish. We gained practical experience using Python libraries such as Pandas, scikit-learn, and PyTorch, while also learning about the importance of preprocessing, model selection, and evaluating performance on unseen data. We also recognized that our relatively small dataset limits how broadly we can apply these findings, and that collecting more responses could help improve future experiments.